The Journal of Open Source Software

pyaudi: A truncated Taylor polynomial algebra toolbox for differentiable intelligence, automatic differentiation, and verified integration applications.

Dario Izzo [1], **Francesco Biscani** [2], **and Sean Cowan** [1]

1 Advanced Concepts Team, European Space Research and Technology Center **2** Skylon Dynamics

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

pyaudi is an open-source Python toolbox developed at the European Space Agency that provides high-order, forward-mode automatic differentiation in a multivariate setting. The toolbox is built on C++ class templates, exposed to Python via pybind11, and, at its core, implements the algebra of truncated Taylor polynomials. This design allows the underlying generalized dual number type to act as a seamless drop-in replacement for scalar types such as floats, while operator overloading ensures that derivatives are propagated automatically. The C++ code base audi can also be used directly and allows for greater flexibility in the instantiation of the algebra over different fields.

All standard mathematical operations are supported, leveraging the nilpotency of exponentiation in the algebra of truncated Taylor polynomials. In essence, within a truncated algebra $\mathcal{A}^n$, any polynomial $p \in \mathcal{A}^n$ with vanishing constant term evaluates to zero when raised to a power greater than the truncation order ($m > n$). This property enables efficient and exact computation of derivatives to arbitrary order while maintaining the simplicity of standard numerical code.

On top of the algebra of truncated Taylor polynomials, pyaudi also offers an implementation of Taylor models (Makino, 1998), which combines truncated Taylor polynomials with an interval bounding their truncation error as well as a number of miscellaneous algorithms useful for applications in differential intelligence, high-order automatic differentiation, verified integration and more.

Statement of need

pyaudi enables users to compute and manipulate order-n Taylor expansions of generic computational graphs, while also providing rigorous bounds on the truncation error through their associated Taylor models. These Taylor representations of program outputs can be exploited in a variety of ways, including fast Monte Carlo simulations, rigorous uncertainty analysis, local inversion of output–input relations, and high-order sensitivity studies. The package implements the high-order automatic differentiation methodology originally developed by Berz and Makino ((Berz et al., 2014), (Makino, 1998)), while introducing novel implementation details in polynomial multiplication routines and in the bounding of Taylor models.

Existing libraries with similar capabilities

As of the time of writing, there are two main open source libraries that allow to perform similar computations to those allowed by pyaudi. The first one is the C/C++ library DACE (Massari

et al., 2018) implementing the full differential algebra of truncated Taylor polynomials with float coefficients. Unlike pyaudi, DACE relies on a polynomial multiplication routine that makes extensive use of memory for the storage of monomial coefficients. As discussed in the comparison reported below, this approach gives DACE an advantage for single evaluations at lower orders, with the benefit diminishing as computations are performed in batches and at high orders.

A second relevant project are the Julia libraries TaylorSeries.jl and TaylorModels.jl (Benet et al., 2019) providing implementations of Taylor models to compute rigorous bounds on generic Taylor series. However, their underlying approach differs substantially from that of pyaudi, and preliminary comparisons presented here indicate that pyaudi can significantly outperform these libraries in the practical cases tested.

In the field of machine learning, in recent years a plethora of automatic differentiation toolboxes have risen tailored at the efficient execution of machine learning tasks and optimization in general. JAX (Bradbury et al., 2018), TensorFlow (Abadi et al., 2015), PyTorch (Paszke et al., 2019) (autograd) are perhaps the most widely adopted. For low order derivatives (first and second order only) these are very efficient and should be used as they implement efficiently reverse mode automatic differentiation which for these tasks is often superior. For higher order derivatives, instead, most implementation suffer greatly and, although some experimental features start to be available (see JAX's jet feature (Bettencourt et al., 2019)), they are still not mature enough.

Key aspects

The main features of pyaudi are:

- **Truncated polynomial algebra**, powered by Obake, a C++ library for symbolic manipulation of sparse multivariate polynomials, truncated power series, and Poisson series. Unlike other packages, which often suffer from severe memory bottlenecks as the polynomial order or the number of variables increases, pyaudi avoids large static memory allocations by adopting a sparse, dynamic approach. This remains memory-efficient at the cost of additional bookkeeping, where sparse polynomials are the area of greatest benefit. The use of templates allows to instantiate the algebra over different fields such as floats, quadruple precision floats, vectorized floats, etc..
- **Vectorized coefficients**, enabling the simultaneous evaluation of identical computational graphs at multiple expansion points. This feature makes it possible to compute high-order derivatives on multiple points, while alleviating the overhead introduced by the sparse bookkeeping of Obake.
- **Taylor models with Bernstein polynomial bounding**, used to enclose the range of multivariate polynomials.
- **Map inversion algorithm**, implementing the method described in (Berz et al., 2014). This feature enables the local inversion of input–output relations arising in generic computational graphs.

Comparison with DACE

The main difference between the DACE library, with respect to automated differentiation capabilities, and pyaudi is to be found in their polynomial multiplication algorithm. We thus focus on that for a preliminary comparison. Our benchmarks run on an AMD EPYC 7702 64-Core Processor with 512GB of RAM and show the relative computational time of the exact same quantities. We compare the polynomial multiplication algorithm in pyaudi with the one implemented in DACE, on a single polynomial multiplication.

We thus intoduce two polynomials of the form:

DRAFT

$$p_1 = (1 + x_1 + x_2 + ... + x_n)^m$$
$$p_2 = (1 - x_1 - x_2 - ... - x_n)^m$$

where $x_i, i = 1..n$ etc. are the variables and m the order. The polynomials are then multiplied and the result of $p_1 p_2$ is timed. For this simple and basic operation, the speed up of pyaudi w.r.t. DACE is reported in the table below (2.0 would mean pyaudi is twice as fast as DACE).

nvars↓ Order→	6	7	8	9	10	11	12	13	14	15
6	0.162	0.269	0.0669	0.181	0.157	0.359	0.593	0.63	1.11	1.16
8	0.0538	0.237	0.147	0.438	0.721	0.992	2.25	3.58	6.49	9.26
10	0.168	0.304	0.43	0.641	1.32	2.9	7.04	11	18.5	27.8

It can be seen that pyaudi is faster from $n + m >\approx 19$ where memory management becomes an issue. At lower orders and number of variables DACE is significantly faster as it is able to exploit an easier memory structure and has no overhead.

Vectorized coefficients

In order to mitigate the bookkeeping overheads of `pyaudi`, vectorized coefficients have been implemented as to allow to perform the same computations over batches, also having in mind potential machine learning applications.

To showcase the resulting performances of such a vectorization, we perform the following computation batching the value c_v of the constant coefficient.

$$p_3 = \left(\frac{c_v + x_1 + x_2 + ... + x_n}{c_v - x_1 - x_2 - ... - x_n}\right)^{10}$$

It is worth noting here that this operation is not representative of actual applications where one is mostly interested in computing higher order derivatives of computer programs. Rather it is selected to isolate the feature we are proposing to benchmark which is batching coefficients. In case of DACE we perform the same computation over the entire batch in a loop.

The results are displayed in the three following tables with differing number of variables.

6 variables

points↓ Order →	1	2	3	4	5	6	7	8	9
16	0.221	0.18	0.314	0.076	0.0731	0.142	0.139	0.185	0.337
64	0.967	0.717	0.162	0.183	0.142	0.325	0.413	0.614	0.985
256	1.59	0.212	0.355	0.273	0.312	0.716	1.14	1.68	2.32
1024	0.341	0.444	0.373	0.336	0.478	1.35	1.56	2.65	3.77
4096	0.752	0.533	0.537	0.486	0.679	1.63	2.29	2.82	4.02

8 variables

points↓ Order →	1	2	3	4	5	6	7	8	9
16	0.433	0.231	0.0446	0.0746	0.0774	0.153	0.222	0.367	0.472
64	1.18	1	0.168	0.174	0.195	0.464	0.764	0.978	1.38
256	3.93	0.405	0.261	0.345	0.575	1.28	1.57	2.08	2.95
1024	1.23	0.834	0.468	0.836	1.04	2.05	2.45	3.66	4.3
4096	2.14	0.878	1.03	1.43	1.28	2.24	2.95	3.87	5.11

DRAFT

10 variables

points↓ Order →	1	2	3	4	5	6	7	8	9
16	0.376	0.228	0.0348	0.0705	0.0806	0.177	0.246	0.376	0.586
64	1.05	0.167	0.185	0.189	0.289	0.517	0.832	1.27	1.97
256	3.37	0.489	0.337	0.435	0.796	1.4	2.21	3.34	4.87
1024	1.07	0.816	0.624	1.03	1.35	2.82	4.54	6.7	10
4096	2.36	0.983	1.2	1.35	2.04	3.94	6.36	9.72	15.2

It can be seen that, from batches of size 64 onwards, pyaudi becomes competitive with respect to DACE and significanlty faster at higher orders. Clearly these results are only indicative of the specific computation selected and different sparsities and computations will result in rather different speedups. The tables above are nonetheless useful to establish a trend which remains true in general: applications where very high derivation orders or multiple expansion points need to be computed, will benefit from pyaudi algorithmic implementations.

Comparison with TaylorModels.jl

We here test the performance of the implementation of Taylor models in pyaudi against the Julia package TaylorModels.jl. To perform the comparison we use three functions f, g, h: one univariate, one bivariate and one trivariate defined below. We then construct Taylor models of all the variables separately and time the evaluation of the corresponding Taylor model. The comparison is made on a single CPU machine.

$$
\begin{aligned}
f(x) =& x(x-1.1)(x+2)(x+2.2)(x+2.5)(x+3)\sin(1.7x+0.5) \\
g(x,y) =& \sin(1.7x+0.5)(y+2)\sin(1.5y) \\
h(x,y,z) =& \frac{4\tan(3y)}{3x + x\sqrt{\frac{6x}{-7(x-8)}}} - 120 - 2x - 7z(1+2y) \\
& - \sinh\left(0.5 + \frac{6y}{8y+7}\right) + \frac{(3y+13)^2}{3z} - 20z(2z-5) \\
& + \frac{5x\tanh(0.9z)}{\sqrt{5y}} - 20y\sin(3z)
\end{aligned}
$$

Dimension	Package	Remainder Bound (Order 1)	Remainder Bound (Order 15)	Speed Comparison
f(x)	TaylorModels.jl	1e+2	1e-5	~1–1.5× faster than pyaudi
	pyaudi	1e+2	1e-5	~1–1.5× slower than TaylorModels.jl
g(x, y)	TaylorModels.jl	1e+1	1e-6	Slower: pyaudi is 5× faster (order 3), 15× faster (order 15), 7800× faster (order 1, edge case)
	pyaudi	1e+1	1e-6	Faster (see above)
h(x, y, z)	TaylorModels.jl	1e+0	1e-11	Slower: pyaudi is 8× faster (order 3), 155× faster (order 15), 13000× faster (order 1, edge case)
	pyaudi	1e-1	1e-17	Faster (see above)

DRAFT

Dimension	Package	Remainder Bound (Order 1)	Remainder Bound (Order 15)	Speed Comparison

In the table above, a clear trend can be seen both in terms of speed and accuracy. For univariate Taylor models, TaylorModels.jl and `pyaudi` have similar performances. At two dimensions, while the remainder bounds are comparable in size, `pyaudi` is significantly faster, with the speedup increasing with the order of the polynomial. At three dimensions, `pyaudi` produces significantly tighter bounds and is again significantly faster, with the speedup increasing with the order of the polynomial.

Ongoing research

(Acciarini et al., 2024) extended the principle of numerical continuation to find solutions in non-linear dynamical systems in the space domain to the moments of a probability density function, using `pyaudi` to calculate the high-order derivatives in the Circular Restricted Three-Body Problem (CR3BP). Similarly, `pyaudi` was used in (Acciarini et al., 2025) for various test cases involving non-Gaussian uncertainty distributions to calculate the high-order derivatives. (Caleb & Lizy-Destrez, 2020) did work on exploring Differential Algebra (DA) for accelerating Monte Carlo simulations, which is enabled by libraries such as `pyaudi`. For on-board applications, `pyaudi` was used in (Burnett & Topputo, 2025) to implement convex guidance algorithms by describing the solutions using DA. (Izzo et al., 2017) explored Cartesian Genetic Programming and made it differentiable with high-order automatic differentiation, enabled by `pyaudi`.

Acknowledgement of financial support

No financial support was provided for the development of this software.

Bibliography

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://www.tensorflow.org/

Acciarini, G., Baresi, N., Lloyd, D. J., & Izzo, D. (2024). Stochastic continuation of trajectories in the circular restricted three-body problem via differential algebra. *arXiv Preprint arXiv:2405.18909*.

Acciarini, G., Baresi, N., Lloyd, D. J., & Izzo, D. (2025). Nonlinear propagation of non-gaussian uncertainties. *Journal of Guidance, Control, and Dynamics*, *48*(4), 903–913.

Benet, L., Forets, M., Sanders, D., & Schilling, C. (2019). TaylorModels. Jl: Taylor models in julia and their application to validated solutions of ODEs. In *SWIM* (pp. 15–16).

Berz, M., Makino, K., & Wan, W. (2014). *An introduction to beam physics*. Taylor & Francis.

Bettencourt, J., Johnson, M. J., & Duvenaud, D. (2019). Taylor-mode automatic differentiation for higher-order derivatives in JAX. *Program Transformations for ML Workshop at NeurIPS 2019*.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

DRAFT

Burnett, E. R., & Topputo, F. (2025). Rapid nonlinear convex guidance using a monomial method. *Journal of Guidance, Control, and Dynamics*, *48*(4), 736–756.

Caleb, T., & Lizy-Destrez, S. (2020). Can uncertainty propagation solve the mysterious case of snoopy? *International Conference on Uncertainty Quantification & Optimisation*, 109–128.

Izzo, D., Biscani, F., & Mereta, A. (2017). Differentiable genetic programming. *European Conference on Genetic Programming*, 35–51.

Makino, K. (1998). *Rigorous analysis of nonlinear motion in particle accelerators* [PhD thesis]. Michigan State University.

Massari, M., Di Lizia, P., Cavenago, F., & Wittig, A. (2018). Differential algebra software library with automatic code generation for space embedded applications. In *2018 AIAA information systems-AIAA infotech@ aerospace* (p. 0398).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*.

DRAFT